

04_Engineering_Standards.md

Part 1 — Engineering Foundation

Document Information

Property	Value
Project	Aetheris Trade
Document	Engineering Standards
Version	1.0.0
Status	Draft
Owner	Priyanshu Rajpurohit

Purpose

This document defines the official engineering standards for Aetheris Trade.

It establishes:

- Engineering philosophy
- Project structure
- Coding conventions
- Folder organization
- Technology standards
- Development workflow

Every developer and AI assistant must follow these standards.

Engineering Philosophy

The codebase must be:

- Scalable
- Maintainable
- Readable
- Modular
- Secure
- Performant

- Testable
-

Engineering Principles

1. Clean Code
 2. Reusability First
 3. Composition Over Duplication
 4. Type Safety
 5. Performance by Default
 6. Security by Design
 7. Documentation Before Complexity
-

Official Tech Stack

Frontend

- Next.js
 - React
 - TypeScript
 - Tailwind CSS
 - Framer Motion
-

Backend

- Next.js API Routes
 - Prisma ORM
 - PostgreSQL
-

Authentication

- Clerk
-

Charts

- TradingView Lightweight Charts
-

AI

- OpenAI API
-

Deployment

- Vercel
-

Project Structure

```
aetheris-trade/  
|  
├─ app/  
├─ components/  
├─ features/  
├─ lib/  
├─ hooks/  
├─ services/  
├─ store/  
├─ prisma/  
├─ public/  
├─ styles/  
├─ types/  
├─ utils/  
├─ constants/  
├─ middleware/  
├─ tests/  
└─ docs/
```

Folder Responsibilities

app/

- Routes
 - Layouts
 - Pages
-

components/

- Shared UI Components
-

features/

- Business Modules

Examples

- portfolio
 - market
 - ai
 - learning
-

lib/

- Third-party configuration
 - Database
 - API clients
-

services/

- Business logic
 - API communication
-

store/

- Global state management
-

hooks/

- Custom React Hooks
-

utils/

- Utility functions
-

types/

- Shared TypeScript types
-

constants/

- Static values
 - Enums
 - Configurations
-

Naming Conventions

Folders

```
kebab-case
```

Example

```
market-explorer  
paper-trading
```

Files

```
kebab-case
```

Example

```
stock-card.tsx  
portfolio-chart.tsx
```

React Components

```
PascalCase
```

Example

```
StockCard  
PortfolioSummary  
AIChatPanel
```

Functions

```
camelCase
```

Example

```
calculateProfit()  
  
getPortfolioValue()  
  
fetchMarketData()
```

Variables

```
camelCase
```

Example

```
portfolioValue  
  
stockPrice  
  
currentBalance
```

Constants

```
UPPER_SNAKE_CASE
```

Example

```
MAX_PORTFOLIO_SIZE  
  
DEFAULT_BALANCE  
  
API_TIMEOUT
```

Import Order

1. External Libraries
2. Internal Modules
3. Components
4. Hooks
5. Utilities
6. Types
7. Styles

Code Organization Rules

- One component per file.
- One responsibility per component.
- Keep files focused.
- Avoid deeply nested folders.
- Reuse before creating new components.

Documentation Rules

Every exported function should include:

- Purpose
- Parameters
- Return Value

Complex logic should be documented.

Validation Checklist

- Folder structure followed
- Naming conventions followed
- Tech stack unchanged
- Clean project organization
- Documentation updated

Related Documents

- 09_Design_System.md
- 10_Database_Architecture.md
- 11_API_Architecture.md

Approval Status

Version: 1.0.0

Status: Draft

Next Document

04_Engineering_Standards.md — Part 2